

3D Particle Tracking Python Model User Guide

Sara Cloux^{*1}, Cristobal López¹, Emilio Hernández¹

¹ *IFISC-UIB* (Institute for Cross-Disciplinary Physics and Complex Systems)



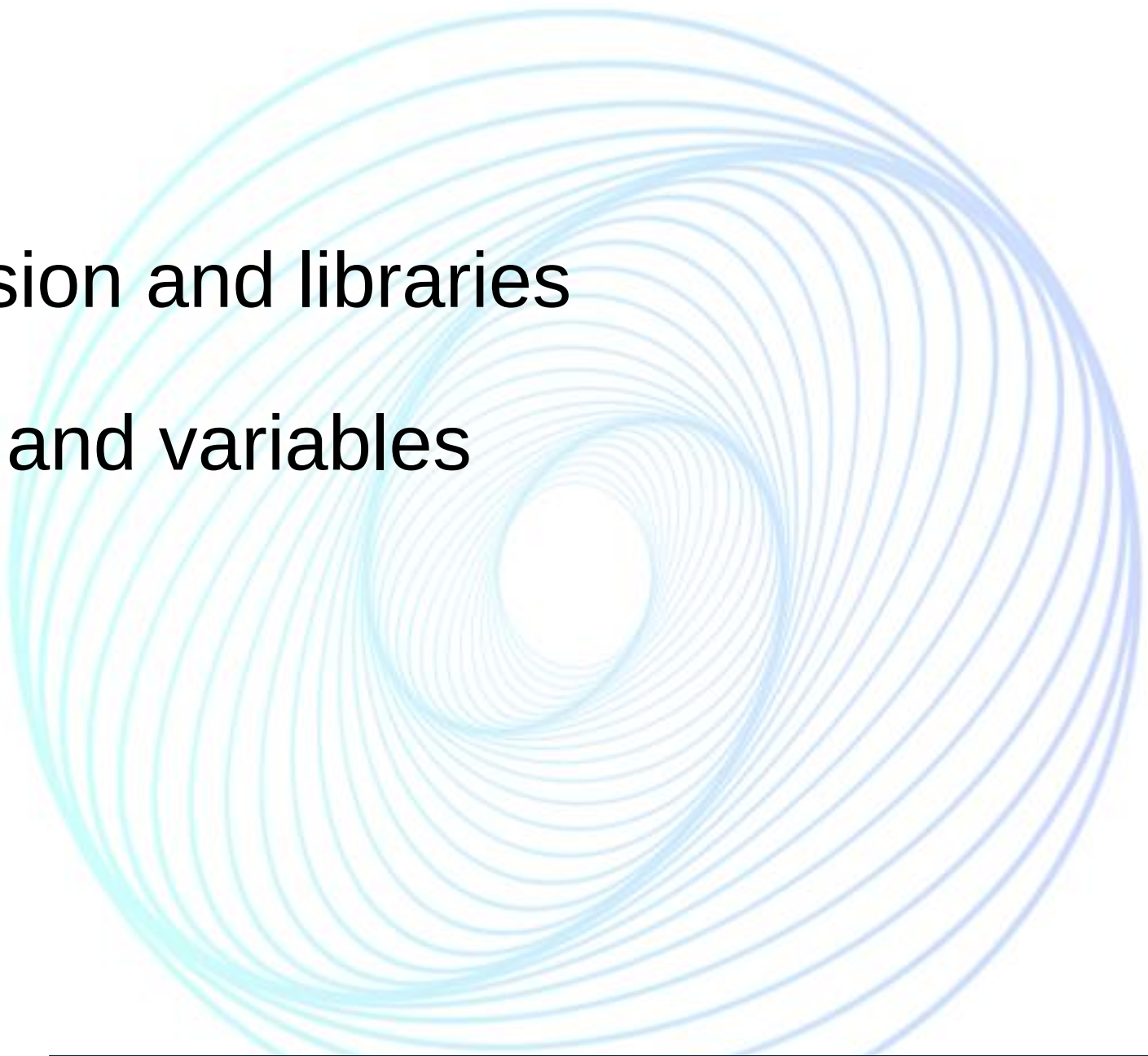
LAMARCA

Lagrangian transport of MARine litter and microplastics
from modeling, analysis and observations in CoAstal waters



3D Particle Tracking Python Model. User Guide

Index



- . Python version and libraries
- . Arguments and variables
- . Functions
- . Outputs

Python version and libraries

We use Python version 3.9, and the following libraries (versions are commented on the right side) are imported within the program:

```
import numpy as np                                # v 1.23
import matplotlib.pyplot as plt                   # v 3.6.2
import xarray as xr                               # v 2023.6.0
from scipy.interpolate import RegularGridInterpolator # v 1.10
from scipy.integrate import solve_ivp              # v 1.10
import pandas as pd                               # v 1.5.3
import csv
import datetime
import argparse
from datetime import datetime, timedelta
```

Arguments and variables

All arguments of this program can be passed internally or use an argument parser to pass them externally. In the latter way, the user can choose in each execution the parameters. Here is an example of both ways:

```
# Create the argument parser
# The user can choose the time direction (BACKWARD = -1 or FORWARD = 1) and the
# starting date (YYYY-MM-DDTHH:MM "(2011-01-20T10:30)" format)

parser = argparse.ArgumentParser(description='Simulación con parámetros.')
parser.add_argument('--direction', choices=['-1', '1'], required=True, help='Dirección de la simulación')
parser.add_argument('--date', required=True, help='Fecha de inicio en formato YYYY-MM-DDTHH:MM (2011-01-20T10:30)')

# Parse the command line arguments

args = parser.parse_args()

# Assign the value of '--direction' and '--date' to a variable

time_direction = int(args.direction)
empieza = args.date
```

Arguments and variables

All arguments of this program can be passed internally or use an argument parser to pass them externally. In the latter way, the user can choose in each execution the parameters. Here is an example of both ways:

```
# Define the depth ('k'), time step ('dt_hours') and total time of simulation ('simulation_time')
k = 0.0
dt_hours = 1.0
simulation_time = 25
```

Functions

There are 2 key functions in trajectory calculation: `interpolate_velocity_3D` and `trajectory_ode_3D`.

The function `simulate_trajectories_3D` incorporates both functions to return a dataset with the trajectories depending on the instant `t`, for each trajectory with index `id`. The output format is `netcdf`, but the user can modify this part of the code to define other formats (This function will not be explained in detail, as it simply organizes the output data into a specific format (NetCDF) that was of interest to us. However, the user is free to choose any other format)

Functions. `interpolate_velocity_3D`

`interpolate_velocity_3D` returns the velocity values as a function of the three components of the velocity field (u, v, w) for any point in space that is within the domain of our velocity field and is defined by the coordinates ($t, \text{latitude}, \text{longitude}, \text{depth}$).

Functions. interpolate_velocity_3D

```
# Define the function for interpolate the velocity fields in 3D

def interpolate_velocity_3D(ds, latitudes_deg, longitudes_deg, z_m, t):

    # Retrieve latitudes, longitudes, and times from the dataset
    latitudes = ds[latitude_ds_name].values
    longitudes = ds[longitude_ds_name].values
    times = ds[time_ds_name].values.astype('float64') / 1e9 # Convert to seconds since the epoch
    depth = ds[depth_ds_name].values

    u = ds[uo_ds_name][:,:,:,:].values # Zonal velocity (u)
    v = ds[vo_ds_name][:,:,:,:].values # Meridional velocity (v)
    w = ds[wo_ds_name][:,:,:,:].values # Vertical velocity (w)

    # Create 3D interpolators for u, v, and w velocities
    interpolator_u = RegularGridInterpolator((times, latitudes, longitudes, depth), u, bounds_error=False, fill_value=np.nan)
    interpolator_v = RegularGridInterpolator((times, latitudes, longitudes, depth), v, bounds_error=False, fill_value=np.nan)
    interpolator_w = RegularGridInterpolator((times, latitudes, longitudes, depth), w, bounds_error=False, fill_value=np.nan)

    # Interpolate velocities at the specified latitude, longitude, depth, and time
    interpolated_u = interpolator_u((t, latitudes_deg, longitudes_deg, z_m))
    interpolated_v = interpolator_v((t, latitudes_deg, longitudes_deg, z_m))
    interpolated_w = interpolator_w((t, latitudes_deg, longitudes_deg, z_m))

    # Return the interpolated velocity components
    return interpolated_u, interpolated_v, interpolated_w
```

Functions. trajectory_ode_3D

`trajectory_ode_3D` solves the system of equations `ode` to give the position of each trajectory in `latitude`, `longitude`, `depth` for every time step `t`. It also returns the value of the velocities (u, v, w) at each point.

Functions. trajectory_ode_3D

```
def trajectory_ode_3D(t, state, ds, _=None):

    # Unpack the current state (latitude, longitude, depth)
    lat, lon, z = state

    # Apply a correction factor based on the latitude (for longitude scaling)
    correction = np.cos(lat * (np.pi / 180.))

    # Interpolate velocity at the current position and time (t)
    u_curr, v_curr, z_curr = interpolate_velocity_3D(ds, lat, lon, z, t)

    # Calculate the new latitude, longitude, and depth based on the interpolated velocities
    new_lat = lat + (u_curr * dt)
    new_lon = lon + (v_curr * dt) / correction
    new_z = z + (z_curr * dt_z)

    # Return the updated position (latitude, longitude, depth) and current velocities
    return new_lat, new_lon, new_z, u_curr, v_curr, z_curr
```

Outputs

In our case, the outputs store the position information as a function of the variables latitude, longitude and depth, in coordinates time and id. As attributes, we also store the initial latitude and longitude values, and the shape of the initial mesh they conform.

Outputs

```
<xarray.Dataset>
Dimensions:      (id: 147953, time: 360)
Coordinates:
  * id           (id) int64 0 1 2 3 4 5 ... 147948 147949 147950 147951
147952
  * time         (time) object '2011-02-26T03:30:00' ... '2011-02-
11T04:30:00'
Data variables:
  latitude       (id, time) float64 ...
  longitude      (id, time) float64 ...
  depth          (id, time) float64 ...
Attributes:
  latitude_init: [38.        38.0025 38.005   ... 38.61    38.6125
38.615 ]
  longitude_init: [0.1    0.1    0.1    ... 1.595 1.595 1.595]
  reshape_dimensions: [599 247]
```

Outputs

```
<xarray.Dataset>
Dimensions:      (id: 147953, time: 360)
Coordinates:
  * id           (id) int64 0 1 2 3 4 5 ... 147948 147949 147950 147951
147952
  * time         (time) object '2011-02-26T03:30:00' ... '2011-02-
11T04:30:00'
Data variables:
  latitude       (id, time) float64 ...
  longitude      (id, time) float64 ...
  depth          (id, time) float64 ...
Attributes:
  latitude_init: [38.        38.0025 38.005   ... 38.61    38.6125
38.615 ]
  longitude_init: [0.1    0.1    0.1    ... 1.595 1.595 1.595]
  reshape_dimensions: [599 247]
```